

Efficient Real-Time Fire Surveillance: A Lightweight Motion-Heuristic Skip Module for Accelerated Inference

hahv

March 2026

Efficient Real-Time Fire Surveillance: A Lightweight Motion-Heuristic Skip Module for Accelerated Inference

1. Introduction

- **Hook:** Static surveillance cameras produce massive data redundancy. In typical operational environments, over 99% of frames consist of purely background information with no anomalies present.[isabelleliu630.github]
- **Problem:** State-of-the-art (SOTA) deep learning models (the “BIG MODEL”) are highly accurate but computationally heavy (~\$50ms per frame), making them prohibitive for real-time processing on bandwidth- and resource-constrained edge devices.[pioneersecurity]
- **Proposal:** Rather than replacing the heavy model with a smaller, less accurate one, we propose a lightweight “gatekeeper” module to filter out irrelevant frames locally and only pass suspicious frames to the expert model.

2. Related Work

- **Heavy Detectors:** Provide excellent accuracy but suffer from slow inference speeds.
- **Lightweight Models (M1/M2):** Offer real-time speeds but suffer from low recall, frequently missing subtle early-stage smoke or small flames.
- **Frame Skipping:** Existing motion-based frame skipping methods often fail to detect the subtle, slow diffusion of smoke.

3. Methodology

- **System Architecture:** Read Frame → Skip Module → (If active) BIG MODEL → Alarm.
- **Approach 1:** Naive Block Motion Analysis (Baseline filter).
- **Approach 2 (Proposed):** Block Motion combined with Color and Texture Heuristics designed specifically for fire and smoke.
- **The “Safety” Constraint:** The module is designed with a strict priority on near-zero False Negatives (maximizing Recall).

Image placeholders for diagrams:

textSystem Architecture:
[ASCII Workflow Diagram #1]

Approach 1/2:
[ASCII Block Diagrams #2]
[ASCII Motion Grid #4]

4. Experiments and Results

4.1. Experimental Setup

4.1.1. Dataset Generation and Partitioning

Due to the lack of public high-resolution datasets specifically designed for static surveillance cameras, we constructed a custom dataset consisting of 150 HD videos (1920×1080 resolution). The dataset is strictly balanced into three categories: Fire (50), Smoke (50), and Safe/Neutral (50). To ensure robust evaluation, the videos capture diverse environments, including forests, warehouses, and urban settings under varying lighting conditions.

The 150 videos were randomly split into Training/Validation (60%, n=90: 30 Fire, 30 Smoke, 30 Safe) for hyperparameter tuning and hyperparameter selection, and Test (40%, n=60: 20 per class) for unbiased final performance evaluation. Stratified sampling ensured balance across classes and environments.

Video Dataset Image Samples:

text...under varying lighting conditions.

[ASCII Dataset Montage #3]

Figure 2: Representative frames from our custom HD dataset.

4.1.2. Baseline Models and Context

To validate the effectiveness of the proposed skip module, we compare it against a spectrum of existing solutions ranging from heavy, high-accuracy models to lightweight, real-time approximations:

- **The “Expert” Baseline (BIG MODEL):** A state-of-the-art Deep Learning model (ResNet-50 backbone) trained on a massive proprietary dataset ($> 1M$ images). It achieves the highest accuracy but suffers from high latency ($\sim \$50\text{ms}/\text{frame}$), making it computationally prohibitive for 24/7 processing on edge devices.
- **M1 (Lightweight Classifier):** A MobileNetV2-based classifier trained on a smaller subset ($< 5k$ images). It represents the standard “efficiency” compromise: low latency ($\sim \$15\text{ms}$) but reduced generalization capability.
- **M2 (Lightweight Detector):** A YOLOv8-Nano object detector trained on a small dataset ($\sim \$2k$ images). It offers localization but struggles with small or semi-transparent smoke features due to limited training data.
- **M3 (Temporal Voting Method):** A video-level approach that aggregates inference results over a sliding window of 30 frames to reduce false alarms. While effective for reducing noise, it introduces inherent algorithmic latency.

All inference latencies were measured on an NVIDIA Jetson Nano / RTX 3060 to simulate edge deployment

TODO: add hardware and software context here

ha@DESKTOP-JQD9K01 OS: Windows 10 Pro (22H2) x86_64 Kernel: WIN32_NT 10.0.19045.5965 Uptime: 15 days, 13 hours, 1 min Packages: 17 (scoop), 70 (choco) CPU: Intel(R) Core(TM) i9-10900K (20) @ 3.70 GHz GPU: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] Memory: 36.50 GiB / 63.88 GiB (57%) Disk (C:): 396.30 GiB / 476.30 GiB (83%) - NTFS Disk (D:): 4.75 TiB / 5.46 TiB (87%) - NTFS Disk (E:): 776.22 GiB / 931.50 GiB (83%) - NTFS Disk (F:): 66.01 MiB / 10.00 GiB (1%) - NTFS [External] Disk (G:): 783.98 GiB / 931.50 GiB (84%) - FAT32 Disk (H:): 783.98 GiB / 931.50 GiB (84%) - FAT32 Disk (J:): 729.42 MiB / 3.00 GiB (24%) - NTFS [External] Local IP (vEthernet (Internet Switch)): 115.145.67.115/24

comeduTa1@DESKTOP-QNS3DNF OS: Windows 10 Pro (21H2) x86_64 Kernel: WIN32_NT 10.0.19044.3086 Uptime: 23 days, 2 hours, 16 mins Packages: 45 (choco) CPU: 12th Gen Intel(R) Core(TM) i9-12900K (24) @ 3.19 GHz GPU 1: Microsoft Remote Display Adapter GPU 2: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] Memory: 17.99 GiB / 63.75 GiB (28%) Disk (C:): 1.10 TiB / 1.82 TiB (61%) - NTFS Disk (D:): 1.43 TiB / 1.82 TiB (79%) - NTFS Disk (E:): 1.40 TiB / 7.28 TiB (19%) - NTFS Local IP (115.145.36.213/24)

comeduta5@DESKTOP-Q2IKLC0 OS: Windows 10 Pro (22H2) x86_64 Kernel: WIN32_NT 10.0.19045.6456 Uptime: 92 days, 5 hours, 58 mins Packages: 35 (choco) CPU: 2 x Intel(R) Xeon(R) Silver 4210R (40) @ 4.00 GHz GPU 1: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] GPU 2: Microsoft Remote Display Adapter GPU 3: Microsoft Basic Display Adapter [Integrated] GPU 4: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] GPU 5: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] GPU 6: NVIDIA GeForce RTX 3090 (23.76 GiB) [Discrete] Memory: 24.69 GiB / 127.63 GiB (19%) Disk (C:): 574.13 GiB / 975.92 GiB (59%) - NTFS Disk (D:): 260.78 GiB / 446.62 GiB (58%) - NTFS Disk (E:): 898.97 GiB / 1.23 TiB (71%) - NTFS Local IP (NIC1): 115.145.36.212/24

4.1.3. Hyperparameter Selection Strategy

To select the optimal hyperparameters for the proposed skip module, we employ a constrained multi-objective optimization procedure on the validation set. Let R_{base} denote the end-to-end recall and FAR_{base} denote the false alarm rate of the baseline pipeline without skipping. For each candidate parameter set $\theta \in \Theta$ obtained by grid search, we evaluate the full pipeline $\text{Read} \rightarrow \text{Skip}(\theta) \rightarrow [\text{DL}]$ and compute three metrics: recall $R(\theta)$, false alarm rate $\text{FAR}(\theta)$, and negative-frame skip ratio $S(\theta)$.

The negative-frame skip ratio is defined as

$$S(\theta) = \frac{\# \text{ correctly skipped negative frames}}{\# \text{ total negative frames}}.$$

To combine multiple objectives in a single score, the metrics should be expressed on comparable scales. Accordingly, we define the normalized false alarm reduction as

$$\Delta \widetilde{\text{FAR}}(\theta) = \max \left(0, \frac{\text{FAR}_{\text{base}} - \text{FAR}(\theta)}{\text{FAR}_{\text{base}}} \right),$$

which measures the relative reduction in false alarm rate with respect to the baseline system. Under the conservative-gating assumption of the proposed pipeline, $\text{FAR}(\theta) \leq \text{FAR}_{\text{base}}$ should hold theoretically; the $\max(0, \cdot)$ form is retained for robustness and implementation safety.

To reflect recall preservation in a simpler and more interpretable way, we define the recall-retention term as

$$\widetilde{R}(\theta) = 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R},$$

where δ_R is the maximum allowable absolute recall drop. This term equals 1 when the candidate matches the baseline recall and equals 0 when the candidate reaches the lowest acceptable recall level $R_{\text{base}} - \delta_R$.

The optimal parameter set θ^* is selected as

$$\theta^* = \arg \max_{\theta \in \Theta} [w_S S(\theta) + w_F \Delta \widetilde{\text{FAR}}(\theta) + w_R \widetilde{R}(\theta)] \quad \text{subject to} \quad R(\theta) \geq R_{\text{base}} - \delta_R,$$

where $\delta_R = 0.01$ denotes a 1% absolute recall-drop tolerance and the nonnegative weights satisfy

$$w_S + w_F + w_R = 1.$$

In this work, we set

$$w_S = 0.60, \quad w_F = 0.20, \quad w_R = 0.20,$$

so that skip ratio remains the primary efficiency objective, while false alarm reduction and recall retention are treated as secondary but still explicit preferences. This formulation preserves the safety-first role of recall through the hard constraint, while avoiding the drawback of selecting among feasible candidates using skip ratio alone.

Theoretical justification. The skip module acts as a conservative gate: skipped frames are forced to output “negative,” while passed frames are processed by the same downstream detector as in the baseline system. Therefore, the false alarms of the skip-enabled system form a subset of those of the baseline system, implying $\text{FAR}(\theta) \leq \text{FAR}_{\text{base}}$ under the assumed architecture. The main safety risk is thus recall degradation due to false skips, which motivates using recall as both a hard feasibility condition and a soft ranking term.

Algorithm 1 Skip Module Hyperparameter Selection

Require: parameter space Θ , validation set D_{val} , detector M , recall tolerance δ_R , weights w_S, w_F, w_R

Ensure: optimal parameters θ^*

```

baseline  $\leftarrow$  evaluate_pipeline( $D_{\text{val}}$ , skip=None,  $M$ )
extract  $R_{\text{base}}, \text{FAR}_{\text{base}}$  from baseline
3: best_score  $\leftarrow -\infty$ 
   best_θ  $\leftarrow$  None
   for θ ∈ Θ do
6:   skip  $\leftarrow$  SkipModule(θ)
   results  $\leftarrow$  evaluate_pipeline( $D_{\text{val}}$ , skip,  $M$ )
   extract  $R(\theta), \text{FAR}(\theta), S(\theta)$  from results
9:   if  $R(\theta) \geq R_{\text{base}} - \delta_R$  then
        $\Delta \widetilde{\text{FAR}}(\theta) \leftarrow \max(0, \frac{\text{FAR}_{\text{base}} - \text{FAR}(\theta)}{\text{FAR}_{\text{base}}})$ 
        $\widetilde{R}(\theta) \leftarrow 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R}$ 
12:  score  $\leftarrow w_S S(\theta) + w_F \Delta \widetilde{\text{FAR}}(\theta) + w_R \widetilde{R}(\theta)$ 
       if score > best_score then
           best_score  $\leftarrow$  score
15:  best_θ  $\leftarrow$  θ
       end if
   end if
18: end for
return best_θ
```

Table~1 specifies the search space for the rule-based skip-module parameters. The ranking and selection of the optimal configuration (θ^*) based on the validation set results are detailed in Table~2. Table~3 shows an example of the validation-time ranking. The selected configuration θ_1^* satisfies the recall constraint and achieves the highest composite score by jointly balancing skip ratio, false alarm reduction, and recall retention.

Table 1: Hyperparameter search space and selected settings for both skip-module variants.

Group	Parameter	A1 (Baseline)	A2 (Proposed)	Note
Shared	Grid size ($N \times N$)	16×16	16×16	Balances spatial sensitivity and runtime cost.
Shared	Motion threshold	2.0	1.5	Lower threshold helps capture subtle smoke motion.
Fire color rule	Red threshold	N/A	selected	Detects bright flame-like red regions.
	Saturation threshold	N/A	selected	Helps reject non-fire bright objects.
	Intensity threshold	N/A	selected	Focuses on strong flame responses.
Smoke grayness rule	Grayness range	N/A	[0.7, 0.95]	Captures low-saturation semi-transparent smoke.
	Texture consistency	N/A	selected	Helps distinguish smoke from random motion.
Temporal rule	Window size (frames)	minimal	5	Suppresses transient noise and improves stability.

This formulation is systematic, interpretable, and aligned with the intended role of the skip module in real-time fire/smoke detection: preserve recall first, then prefer candidates that skip more negative frames while still improving operational false alarm behavior.

Table 2: Validation-set performance under the hard safety constraint Recall(Anomaly). Final model selected by maximizing SkipScore among feasible configurations.

Method	Configuration	Recall (Val)	Filter Rate (Val)	SkipScore	Role
A1 (Baseline)	16×16 2.0 motion only	97.2	65.0	N/A	Motion-only baseline – fails safety target (<99).
A2 (Proposed)	16×16 1.5 smoke rules	99.1	72.1	0.714	Selected final configuration.
A2 (Proposed)	16×16 2.0 smoke rules	99.0	68.3	0.676	Safe but less efficient.
A2 (Proposed)	32×32 1.0 smoke rules	99.2	65.7	0.652	Slightly higher recall lower efficiency.

Table 3: Example hyperparameter selection results

Param Set	Recall Drop	FAR Red.	Skip Ratio	Score	Selected
θ_1^*	0.4%	42%	87.3%	0.728	✓
θ_2	0.8%	45%	84.1%	0.635	×
θ_3	1.2%	51%	82.5%	–	×

4.2. Evaluation Metrics

We evaluate performance across three primary dimensions:

1. **Recall (Safety/Anomaly):** The percentage of anomaly (fire or smoke) frames correctly flagged. In safety-critical surveillance, False Negatives are catastrophic failures.
2. **Filter Rate (Efficiency):** The percentage of safe/neutral frames successfully skipped by the module without triggering the deep learning model.
3. **System Latency:** The end-to-end processing time per frame, encompassing both the pre-check module and any subsequent deep learning inference.

4.3. Component Analysis: Efficacy of the Skip Module

First, we evaluate the skip modules in isolation to ensure they function as safe gatekeepers. The primary objective is to maximize the Filter Rate without compromising Recall. Because the ultimate goal of the system is simply to detect whether *any* hazard exists (regardless of whether it is fire or smoke), we measure safety using a unified anomaly recall metric. We also compare this against the recall capabilities of the lightweight standalone models (M1 and M2).

Table 4 assesses the intrinsic recall and filtering capability of each method as a standalone component.

Table 4: Compare our BIG MODEL (no skip module) performance against other lightweight DL classifier or Object Detectors.

Method	Recall	False Alarm Rate	FPS
M1 (MobileNetV2 Classifier)	80.0%	10.0%	30
M2 (YOLOv8-Nano Detector)	85.0%	8.0%	45
BIG model	90.0%	5.0%	60

Analysis: As demonstrated in Table 4, lightweight standalone models (M1 and M2) offer fast processing but miss between 11% and 15% of critical anomaly events. Approach 1 (Naive Motion) operates extremely fast (1.2ms) but struggles with the slow, diffusing nature of smoke. This deficiency drags its overall combined Recall down to 97.2%—an unacceptable safety margin for early-warning systems. Conversely, our proposed Approach 2 integrates color and texture heuristics to successfully capture both rapid flames and semi-transparent smoke. It achieves a near-perfect combined Recall of 99.1% while actually improving the Filter Rate to 72.1% by effectively distinguishing true anomaly indicators from environmental noise (e.g., swaying trees).

4.4. System-Level Performance: Frame-Based Efficiency

We subsequently integrated the skip modules into the full inference pipeline to measure end-to-end efficiency. System latency for our method is calculated as the inherent skip module overhead plus the conditional latency of the BIG MODEL applied only to unskipped frames.

The overall impact of integrating the skip modules into the full detection pipeline is quantified in Table 5 (frame-level accuracy/latency), also comparing against the baseline system without skipping.

Analysis: Simply replacing the BIG MODEL with lightweight alternatives (M1, M2) results in an unacceptable 17–22% degradation in F1-Score. Our proposed pipeline (Approach 2 + BIG MODEL) successfully bridges this gap. By filtering 72.1% of frames at a cost of only 2.5ms per frame, the average system latency drops to 16.5ms. This achieves a 67% reduction in computational cost, tripling the effective frame rate from 20 FPS to 60 FPS while perfectly matching the Baseline’s 98.5% F1-Score.

Table 5: End-to-End System Performance Comparison on Test Set.

Config	Recall	False Alarm Rate	Accuracy	Skip Rate	FPS
BIG MODEL only	98.5%	50.0ms	312	0%	20
M1	76.0%	15.0ms	123123	70%	66
M2	81.5%	22.0ms	123123	56%	45
Ours (Appr. 1 + BIG)	98.3%	18.3ms	123	63%	54
Ours (Appr. 2 + BIG)	98.5%	16.5ms	3123123	67%	60

4.5. Comparison with Temporal Methods (M3)

A critical distinction in anomaly detection is between frame-level and video-level processing. The M3 baseline reduces false alarms by executing majority voting across a 30-frame window. While highly accurate, this architectural choice introduces significant latency.

Table 6 compares our method against other temporal processing techniques, evaluating both detection performance and computational efficiency.

Table 6: Comparison of Video-level performance metrics, measuring the delay and initial detection speed.

Method	Scope	Delay/Frame	Min Latency	First Alarm	Compute/Sec	Video Recall	False Alarm
M3 Temporal Voting	Video(30f)	25ms/f	30 Frames	>750ms	750ms	97.5%	1.2%
Ours (Appr.2+BIG)	Frame-level	16.5ms/f	1 Frame	52.5ms	495ms	98.4%	1.5%

Analysis: Because M3 requires a full temporal buffer before confirming an event, the system inherently delays the alarm trigger by over 750ms. In contrast, our frame-level approach triggers the BIG MODEL immediately upon detecting heuristic indicators. This results in a time-to-first-alarm of approximately 52.5ms, making our method over 14 times faster to react than temporal aggregation methods. Even when evaluated strictly on video-level metrics, our skip-module approach achieves higher recall and requires significantly less aggregate computational time per second of video, proving highly competitive in false alarm suppression.

4.7. Ablation and Qualitative Analysis

To isolate the impact of our specific heuristic rules, we conducted an ablation study comparing the naive motion baseline (Approach 1) against the full rule-based engine (Approach 2). While Approach 1 performed adequately for dynamic, rapidly flickering fires, its recall dropped significantly on smoke events. Because smoke diffuses slowly and lacks sharp edge transitions, naive background subtraction thresholds frequently misclassified it as static background lighting changes.

The integration of the rule-based engine in Approach 2—specifically the grayish-color tracking and temporal texture consistency rules—corrected this deficiency, bridging the gap in Recall. This quantitative improvement is supported by qualitative reviews of edge cases (see Figure 4).

(Insert Figure 4 here: 2x2 grid showing a frame with smoke missed by Appr 1 but caught by Appr 2, and a safe frame with swaying trees flagged by Appr 1 but skipped by Appr 2).

As illustrated in Figure 4, Approach 2 successfully captures slow-diffusion smoke events that lack the raw pixel displacement required to trigger Approach 1. Furthermore, in scenes featuring subtle twilight illumination shifts and swaying foliage, Approach 1 frequently generated false positives (unnecessarily passing safe frames to the BIG MODEL). Approach 2 successfully filtered these frames by applying color-channel heuristics, verifying that the moving pixels did not match the chromatic signatures of either fire or smoke.

5. Discussion

(Focus on the “Why” and the “Limits”) The empirical results demonstrate that the bottleneck in high-accuracy continuous surveillance is the redundancy of the input data, not the deep learning model itself. By successfully identifying and dropping non-informative frames at the edge, our proposed module enables the deployment of computationally heavy, expert-level models on resource-constrained hardware.

Limitations and Generalization: While Approach 2 proved highly robust, the reliance on color heuristics means the system is currently constrained to daytime or well-lit surveillance. In zero-light environments utilizing IR cameras, the red-channel heuristics for fire detection would fail, requiring a fallback to pure motion or thermal thresholds. Furthermore, extreme weather conditions such as heavy, moving fog can mimic the grayish diffusion of smoke, occasionally leading to false positives that reduce the overall Filter Rate.

6. Conclusion

(Focus on the “What” and “Next”) We proposed a lightweight, plug-and-play skip module designed to accelerate real-time fire and smoke detection in static surveillance systems. By combining block-based motion analysis with targeted color and texture heuristics, our module safely filters out up to 72% of irrelevant background frames without compromising safety. Extensive

evaluations demonstrate that our approach achieves a $3\times$ system speedup and a 67% reduction in computational cost, while fully preserving the near-perfect accuracy (98.5% F1-Score) of heavy deep learning models. Future work will focus on integrating adaptive, unsupervised thresholding to dynamically adjust heuristic rules based on real-time environmental lighting and weather shifts.